SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct
I **Ch.Tharunika Reddy/192324307** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student:

Ch.Tharunika Reddy
Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy.
   How can encryption and access control protect sensitive backup data?
   Explain the cost advantages compared to traditional storage systems.
   Provide an example of a disaster recovery use case.

3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
   Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

Solutions:

1. Auto-Scaling and Load Balancing During Seasonal Sales

When a company experiences unpredictable traffic spikes during seasonal sales, cloud auto-scaling and load balancing help maintain application performance and availability.

1. **Auto-scaling:** Auto-scaling automatically increases or decreases the number of cloud servers based on demand. During a sale, more servers are added to handle increased traffic. After the sale, unnecessary servers are removed to reduce costs.
2. **Load balancing:** A load balancer distributes incoming user requests across multiple servers. This prevents a single server from becoming overloaded and improves application response time.
3. **Reduction of downtime:** If one server fails or becomes overloaded, traffic can be directed to healthy servers. This helps keep the website available.
4. **Role of monitoring tools:** Monitoring tools track CPU utilization, memory usage, network traffic, response time, and error rates. These metrics can trigger auto-scaling actions and alert administrators to problems.
5. **Real-world example:** Consider an e-commerce website such as Amazon during a festival sale. When thousands of customers visit the website to purchase products, auto-scaling adds servers and the load balancer distributes requests among them.

Key terms used in auto-scaling

Horizontal scaling (scale out and scale in): This means adding more servers when demand rises and removing them when demand falls. It is the usual method for web applications, for example going from 4 to 12 web servers at the start of a sale.

Vertical scaling (scale up and scale down): This means giving one server more CPU or memory. It usually needs a restart and has an upper limit, so it is less suitable for sudden spikes.

Elasticity: This is the ability of the cloud to grow and shrink resources automatically and quickly as the workload changes, so capacity follows the traffic curve through the day.

Health check: The load balancer sends a regular test (for example to a /health page every 10 seconds) to each server. A server that fails the test is removed from rotation until it recovers.

Cooldown period: A short waiting time (for example 5 minutes) after a scaling action so that the system does not add and remove servers too frequently.

How the architecture works (flow chart)

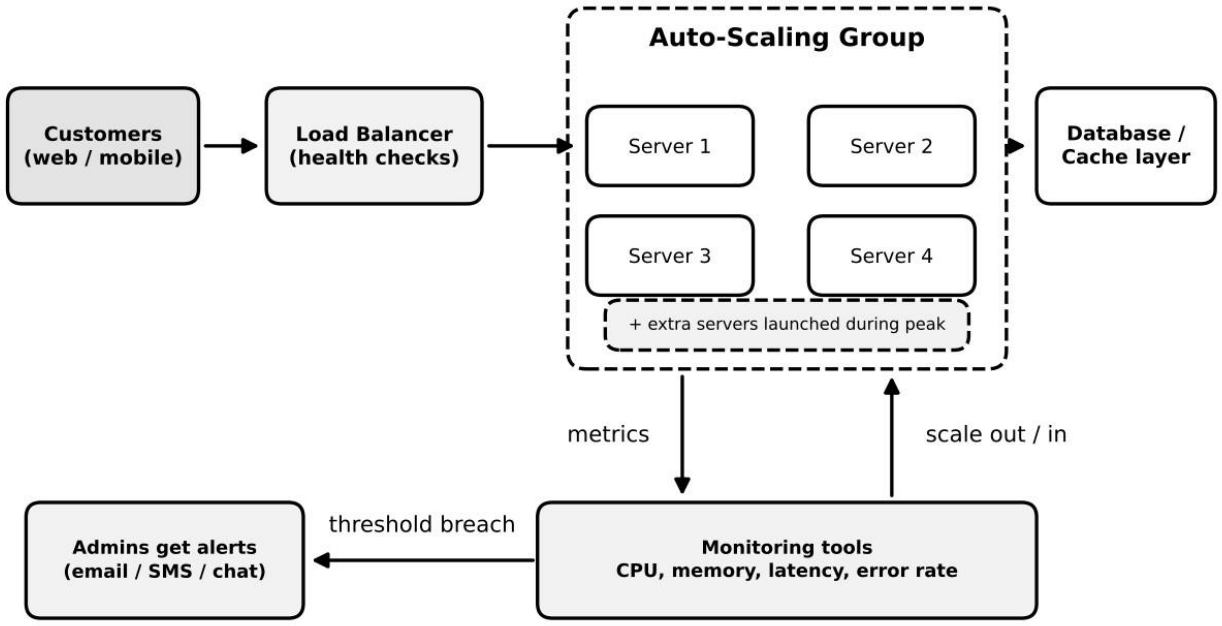


Figure 1.1: Load balancer, auto-scaling group and monitoring working together

Flow of a request: (1) A customer opens the shopping website. (2) The request reaches the load balancer. (3) The load balancer checks which servers are healthy and forwards the request to one of them. (4) The server reads or writes data in the database or cache and returns the page. (5) Monitoring tools collect metrics from every server. (6) When a metric crosses a threshold, the scaling policy launches new servers (scale out) or removes extra servers (scale in), and administrators are alerted if needed.

Common load-balancing methods

Round robin sends each new request to the next server in turn and works well when all servers have similar capacity. **Least connections** sends the request to the server that currently has the fewest active connections, which is better when requests take different amounts of time, such as search or checkout. **Weighted distribution** gives servers with more power a larger share of requests, which helps when the pool contains servers of different sizes. **IP hash or sticky sessions** send the same user to the same server, which is useful when the application keeps session data such as a shopping cart on the server.

Fixed servers versus auto-scaled servers during a sale day

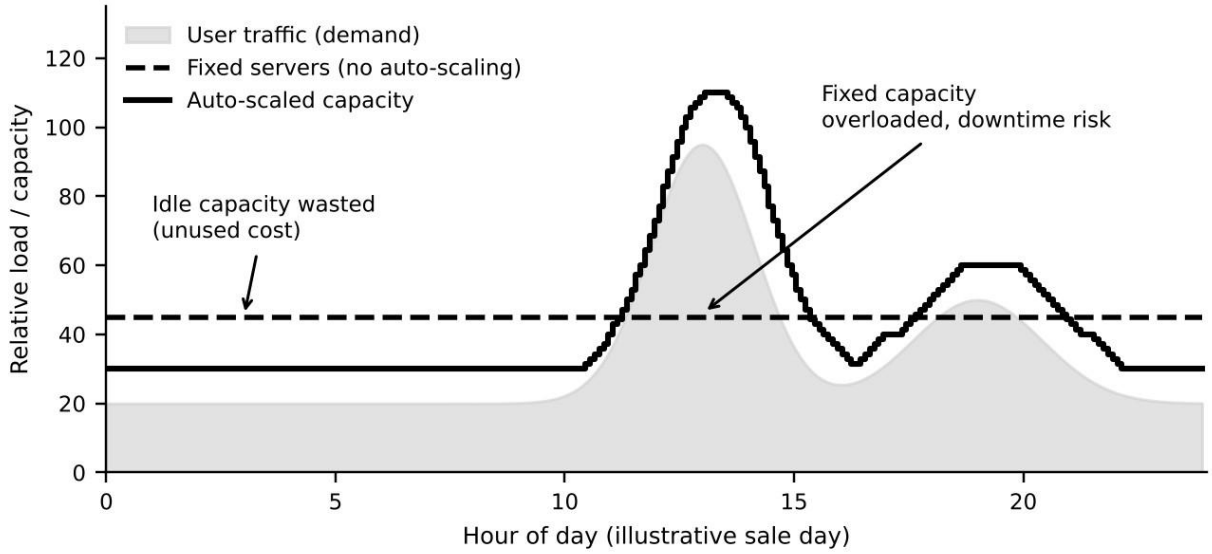


Figure 1.2: Illustrative graph of demand against fixed capacity and auto-scaled capacity (values are for explanation only)

The graph shows that fixed capacity is wasted when traffic is low and is overloaded at the peak, which can cause slow pages or downtime. Auto-scaled capacity rises just ahead of the traffic peaks and falls again afterwards, so users get fast service and the company pays only for what it uses.

Point of comparison	Without auto-scaling and load balancing	With auto-scaling and load balancing
Handling a sudden spike	Servers become overloaded; pages slow down or crash.	New servers are added automatically within minutes.
Failure of one server	Users connected to that server see errors.	Traffic is moved to healthy servers; users barely notice.
Cost	Servers must be bought or rented for the highest possible peak all year.	Extra servers are paid for only while they are needed.
User experience	Slow checkout, failed payments, abandoned carts.	Fast browsing and reliable checkout even at peak.

Monitoring metrics and scaling rules

A typical scaling policy is written as a rule on a monitored metric. For example (values are illustrative): if average CPU utilization stays above 70% for 5 minutes, add 2 servers; if it stays below 30% for 15 minutes, remove 1 server. If response time stays above 2 seconds for three consecutive checks, add servers and alert the operations team. If the HTTP 5xx error rate goes above 2% of requests, send an alert and replace the unhealthy servers. Because the date of a sale is known in advance, scheduled scaling can also raise the minimum number of servers shortly before the sale starts, while the dynamic rules handle unexpected extra traffic.

Festival-sale scenario for an e-commerce website

Before the sale: marketing announces large discounts, so scheduled scaling raises the minimum number of servers and caches are warmed up. **When the sale opens:** thousands of users arrive within a few minutes; the load balancer spreads the requests and auto-scaling adds servers as CPU and request counts rise. **During peak**

hours: monitoring tracks response time and errors while the extra servers keep pages fast. **If a server fails:** the health check detects it, the load balancer stops sending traffic to it, and auto-scaling launches a replacement. **After the sale:** traffic returns to normal and the extra servers are removed, which reduces cost.

Examples of cloud services: Amazon Web Services offers Elastic Load Balancing, EC2 Auto Scaling and CloudWatch; Microsoft Azure offers Azure Load Balancer, Virtual Machine Scale Sets and Azure Monitor; Google Cloud offers Cloud Load Balancing, Managed Instance Groups and Cloud Monitoring.

Conclusion: Auto-scaling and load balancing improve availability, reduce downtime, maintain fast response times, and optimize cloud costs.

2. Secure Cloud Storage for Backup Data

Cloud storage enables organizations to store large amounts of backup data securely, reliably, and cost-effectively.

1. **Durability:** Cloud providers use reliable storage infrastructure and data protection mechanisms to reduce the risk of data loss.
2. **Data redundancy:** Multiple copies of data can be maintained across storage devices or locations. If one device or location fails, another copy can be used to recover the data.
3. **Encryption:** Encryption protects sensitive backup data by converting it into an unreadable form without the appropriate decryption key. Data should be encrypted both at rest and during transmission.
4. **Access control:** Identity and access management (IAM), role-based permissions, and multi-factor authentication help ensure that only authorized users can access backup data.
5. **Cost advantages:** Traditional storage requires organizations to purchase hardware, provide physical space, maintain equipment, and replace outdated devices. Cloud storage reduces these upfront expenses and allows organizations to pay according to their storage usage and selected service tier.
6. **Disaster recovery example:** Suppose a company's office server is damaged by a fire. The company can retrieve its backups from cloud storage and restore its applications and business records on replacement systems.

How a cloud backup flows from the company to the cloud (flow chart)

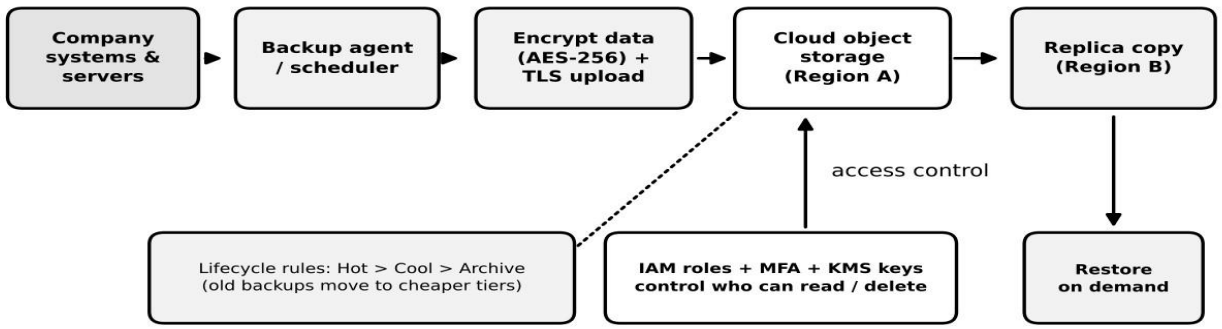


Figure 2.1: Secure backup pipeline with encryption, replication, access control and lifecycle rules

Flow explained: (1) A backup agent or scheduler copies files, databases and server images at fixed times. (2) The data is encrypted before it leaves the company and is sent over an encrypted (TLS) connection. (3) It is stored in cloud object storage in a primary region. (4) The provider keeps additional copies in other devices, zones or regions. (5) IAM roles, multi-factor authentication and key management decide who may read, change or delete backups. (6) Lifecycle rules move older backups to cheaper tiers. (7) When needed, data is restored on demand.

How durability and redundancy are achieved

Cloud providers store each object as several copies on different disks and servers, so the failure of one disk does not cause data loss. Replication across availability zones places copies in separate buildings with their own power and network, which protects against a fire or power cut in one data centre. Cross-region replication copies data to a distant geographic region to protect against a regional disaster. Checksums are verified regularly and damaged copies are repaired automatically, which prevents silent data corruption. Versioning keeps older versions of each file so that accidental deletion or overwriting can be undone, and immutable storage (object lock) prevents backups from being changed or deleted for a set period, which helps against ransomware.

Large providers publish durability targets. For example, Amazon S3 is designed for 99.999999999% (eleven nines) durability of objects. This is a design target, and the customer must still keep correct backup policies.

Layers of security for backup data

Security is applied in layers. **Encryption at rest** (for example AES-256, with keys held in a key management service) means stolen disks or files cannot be read. **Encryption in transit** (TLS or HTTPS) stops data from being read while it travels over the network. **Identity and access management** gives each person or application only the permissions needed (least privilege). **Multi-factor authentication** means a stolen password alone is not enough to reach the backups. **Logging and alerts** record every access and change so that suspicious activity can be detected and audited. **Network controls** such as private connections and IP restrictions reduce exposure to the public internet.

Storage tiers and the 3-2-1 backup rule

Cloud storage offers different tiers. The **hot tier** holds the latest backups that may be restored often; it gives immediate access at the highest price. The **cool or infrequent-access tier** holds backups from the last few months at a lower price. The **archive tier** holds long-term records kept for legal or audit reasons; it is the cheapest, but retrieval can take minutes to hours. Lifecycle rules can move data automatically from one tier to the next as it ages.

The **3-2-1 rule** says to keep **3** copies of important data, on **2** different types of storage, with **1** copy stored off-site. Cloud storage provides the off-site copy automatically.

Cost advantages: traditional storage versus cloud storage

Cost factor	Traditional (on-premises) storage	Cloud storage
Initial investment	Large capital expense for disks, tape libraries and servers	No hardware purchase; start with a small subscription
Capacity planning	Must buy for future growth in advance, so some capacity is idle	Grows as data grows; pay for what is stored
Space, power, cooling	Needs a server room, electricity and air-conditioning	Handled by the provider
Maintenance and staff	Hardware repairs and replacement every few years; storage administrators	Provider manages hardware and upgrades
Off-site copy	Needs a second site or tape transport	Built-in replication across zones and regions
Pricing model	Capital expenditure (CapEx)	Operating expenditure (OpEx), pay-as-you-go

Disaster recovery: terms, strategies and the fire example

RPO (Recovery Point Objective) is the maximum amount of data loss, measured in time, that the business accepts. For example, backups every hour mean up to one hour of data may be lost. **RTO (Recovery Time Objective)** is the maximum time the business can be down before service must be restored, for example 4 hours.

Four common DR strategies exist, from cheapest and slowest to most expensive and fastest. **Backup and restore** keeps backups in the cloud and restores them after a disaster (recovery in hours, lowest cost). **Pilot light** keeps a minimal copy of the core system running that is expanded when needed. **Warm standby** runs a smaller but complete version of the system all the time (recovery in minutes). **Multi-site active-active** runs full systems in two or more locations at once, giving almost instant recovery at the highest cost.

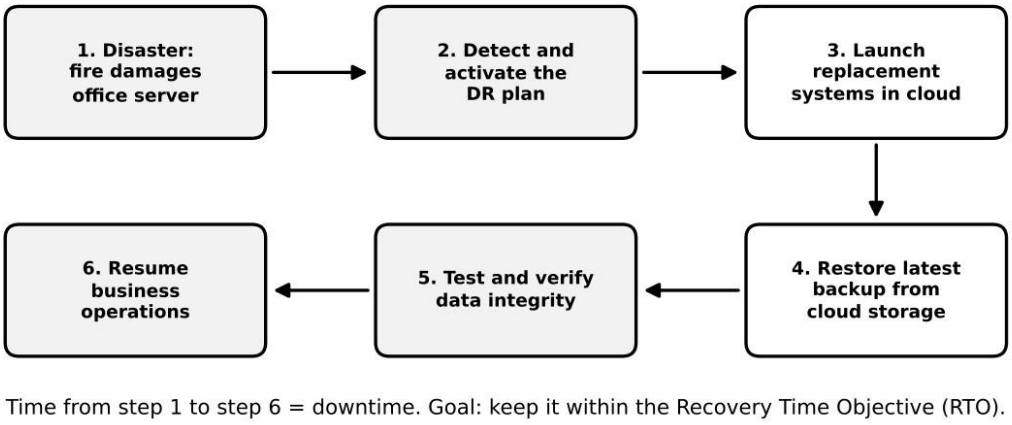


Figure 2.2: Disaster recovery steps for the office-server fire example

Conclusion: Cloud backup provides durability, redundancy, security, scalability, and cost savings while helping organizations recover from disasters.

3. Content Delivery Networks (CDNs) for Global Applications

A Content Delivery Network (CDN) is a geographically distributed network of servers that delivers website content and media to users from locations closer to them.

1.Improved performance: CDNs cache static content such as images, videos, stylesheets, and scripts on distributed servers. Users can access cached content without requesting it from the main server every time.

2.Reduced latency: Latency is the time taken for data to travel between the server and the user. Serving content from a nearby CDN server reduces the distance data must travel and improves response time.

3.Role of edge servers: Edge servers are located closer to end users. They deliver cached content quickly and can support certain edge-computing tasks, improving the user experience across different regions.

4.Global availability: Cloud providers offer geographically distributed regions, availability zones, and CDN infrastructure. These help applications serve users worldwide and improve resilience when individual components experience failures.

5.Real-world example: Consider a video-streaming application such as Netflix. When users in India and the United States watch the same popular movie, CDN servers can deliver video content from nearby locations instead of requiring every request to reach a distant central server.

How a CDN serves a request (flow chart)

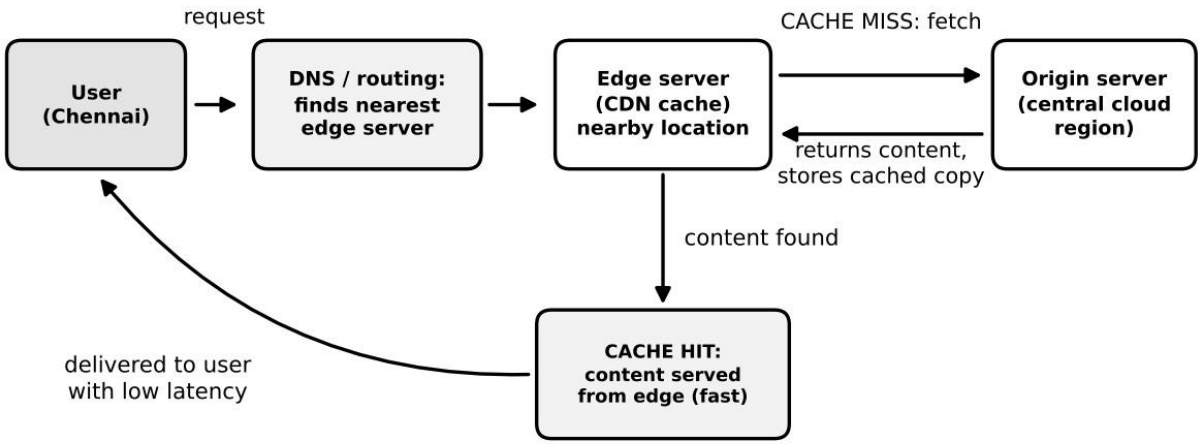


Figure 3.1: Request path through DNS routing, edge server (cache hit or miss) and origin server

Flow explained: (1) A user opens the app or website. (2) DNS or the CDN routing system finds the edge server nearest to the user. (3) If the edge server already has the content (cache hit), it sends it immediately. (4) If it does not (cache miss), the edge server fetches the content once from the origin server, stores a copy for a set

time (TTL), and sends it to the user. (5) The next users in that region receive the cached copy without contacting the origin.

Effect of a CDN on distance and latency

Factor	Without CDN	With CDN
Where content comes from	A single central origin server, possibly on another continent	Edge server in or near the user's city
Network distance	Long; many network hops	Short; fewer hops
Latency (illustrative)	Often 150 to 300 ms or more for distant users	Often 10 to 50 ms for cached content
Load on origin server	Every request reaches it	Only cache misses reach it
Traffic spikes	Origin may be overloaded	Edge servers absorb most of the traffic

Main features of a CDN

Caching with TTL stores copies of images, videos, scripts and style files at the edge for a chosen time (time-to-live). **Cache invalidation** removes or refreshes an old file when a new version is released, so users do not see outdated content. **Compression and optimisation** reduce download size by compressing files and delivering modern image and video formats. **TLS at the edge** encrypts the connection close to the user and reduces the time needed to set it up. **DDoS protection** absorbs large floods of malicious traffic at the edge before they reach the origin, and **dynamic acceleration** uses optimised network routes to speed up content that cannot be cached, such as API calls.

How cloud providers support global availability

A **region** is a geographic area (for example Mumbai, Frankfurt or Virginia) that contains several data centres; placing the application in a region close to users reduces latency and helps meet local data laws. An **availability zone** is an isolated data-centre group inside a region with its own power and network, so the application stays online if one zone fails. **Edge locations** (points of presence) are small sites near large cities used by the CDN and DNS services for fast delivery. **Global load balancing or latency-based DNS** routes each user to the nearest or healthiest region and gives automatic failover. **Multi-region replication** copies data and application versions to more than one region, which protects against a whole-region outage.

Real-time streaming and gaming applications

Video streaming (for example Netflix): the CDN delivers video segments, thumbnails, subtitles and app files. Popular videos are stored near viewers; Netflix places its own cache servers inside many internet providers' networks, so playback starts quickly with less buffering. Streaming can tolerate moderate delay because a short buffer hides small delays, and adaptive bitrate streaming changes video quality to match the viewer's speed. Edge servers can handle millions of viewers at the same time.

Online gaming: the CDN delivers game patches, updates, maps, textures and downloadable content, which are large files that benefit from nearby servers. During play, latency matters much more; delays above about 100 ms are noticeable, so players connect to game servers in the nearest region and matchmaking places players in the same or nearest region. Game servers are started in regions where players are online.

Checklist for deploying a mobile application globally

(1) Host the back end in more than one region, close to major user groups, to reduce latency and give failover.

(2) Serve images, videos and static files through a CDN for faster loading and lower server load.

(3) Use globalload balancing or latency-based DNS to send each user to the best region.

(4) Replicate databases or use regional databases to keep data available and close to users.

(5) Monitor latency and errors in each region to find slow regions early.

(6) Follow local data-protection rules so the application stays legally compliant.

Conclusion: CDNs and edge servers reduce latency, improve loading speed, reduce pressure on origin servers, and provide a smoother experience for global users.

4. Regulatory Compliance and Data Security in Cloud Computing

Financial institutions handle sensitive information such as account details, personal information, and financial transactions. Cloud providers offer security features and compliance support to help protect this information.

1. **Regulatory compliance:** Cloud providers maintain security controls, certifications, and compliance documentation that help organizations meet applicable regulatory requirements. However, the company remains responsible for ensuring that its own cloud usage complies with relevant regulations.

2. **Auditing:** Audit reports and security assessments help organizations verify that appropriate controls are in place. Regular audits identify weaknesses and help demonstrate compliance.

3. **Logging:** Logging records activities such as user logins, file access, configuration changes, and financial transaction events. These records help investigators trace suspicious activities.

4. **Monitoring:** Continuous monitoring detects unusual behavior, unauthorized access attempts, and suspicious transaction patterns. Alerts allow security teams to respond quickly.

5. **Encryption:** Encryption protects sensitive information both at rest and in transit. Secure key management helps prevent unauthorized decryption.

6. **Data sovereignty:** Data sovereignty refers to the legal requirements associated with the country or jurisdiction in which data is stored and processed. The company should select appropriate cloud regions, configure data storage locations, and verify backup, replication, and processing arrangements against applicable laws.

7. **Real-world example:** Suppose a bank processes online money transfers. It can encrypt transaction records, restrict access to authorized employees, record all important activities, and monitor suspicious transactions. It must also ensure that customer data is stored and processed in locations permitted by applicable regulations.

Security and compliance flow for an online money transfer (flow chart)

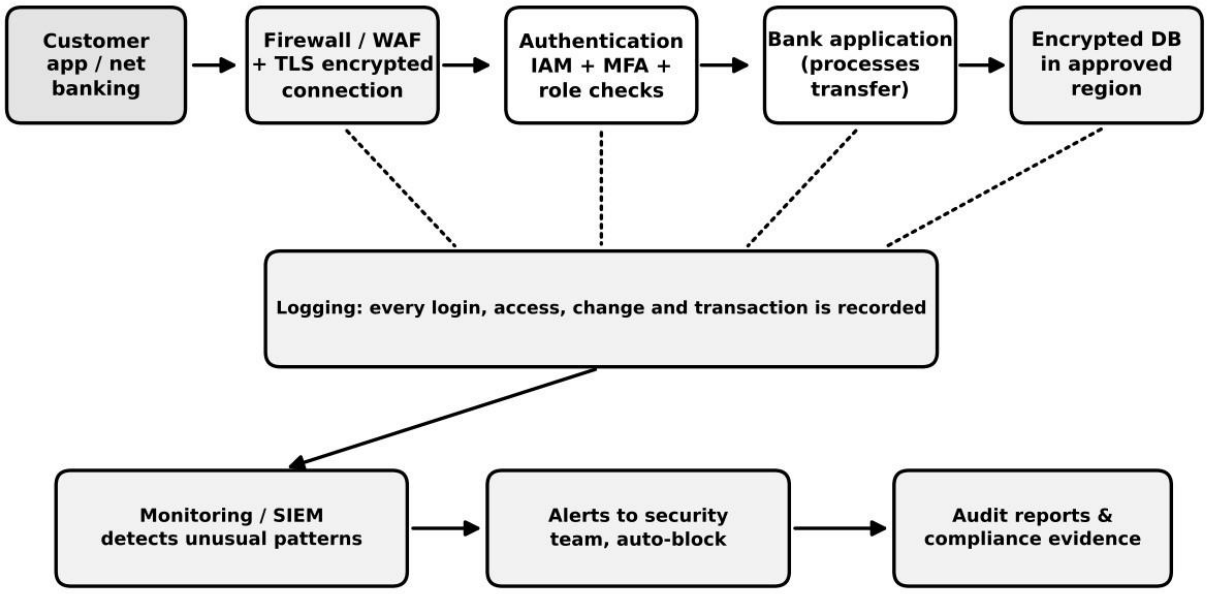


Figure 4.1: Protection of a financial transaction from customer to database, with logging, monitoring and audit

Flow explained: (1) The customer starts a transfer in the banking app. (2) The connection is protected by TLS and checked by a firewall or web application firewall. (3) The user is authenticated with a password plus multi-factor authentication, and role checks decide what the user may do. (4) The bank application processes the transfer. (5) The record is saved in an encrypted database in an approved region. (6) Every step is written to logs. (7) Monitoring tools analyse logs and raise alerts for unusual patterns. (8) Logs and reports become evidence for auditors and regulators.

Shared responsibility model

Area	Cloud provider is responsible for	Financial institution is responsible for
Physical security	Data centre buildings, power, hardware and network equipment	Choosing suitable, certified providers
Infrastructure	Security of the underlying compute, storage and networking	Configuring services securely (for example not leaving storage public)
Data	Provides encryption tools and key services	Classifying data, deciding what is stored and turning encryption on
Identity and access	Provides IAM and multi-factor authentication features	Creating users and roles, enforcing least privilege, reviewing access
Compliance	Maintains certifications and shares audit reports	Proving that its own use of the cloud meets the law

Common standards and regulations

PCI DSS protects payment-card data and is required for organisations that store, process or transmit card data. **ISO/IEC 27001** sets requirements for an information security management system and shows that security is managed in a structured way. **SOC 2** reports on the security, availability and confidentiality controls of service providers, and customers can request the report from the cloud provider. **GDPR** protects the personal data of people in the European Union and limits transfers of that data abroad. In India, the **DPDP Act 2023** covers personal data protection, and RBI directions require payment-system data to be stored in India, which affects the choice of cloud region. Provider certifications help, but the institution must still check which regulations apply to it, because rules differ by country and type of service.

Roles of logging, monitoring and encryption

Logging records who did what, when and from where, using tools such as AWS CloudTrail, Azure Activity Log or Google Cloud Audit Logs; it gives evidence for investigations and audits. **Monitoring** watches logs and metrics continuously and raises alerts (for example CloudWatch, Azure Monitor or a SIEM system), which gives fast detection of fraud and attacks. **Encryption at rest** (AES-256 with KMS or hardware security modules) protects stolen disks and database copies, and **encryption in transit** (TLS 1.2 or higher, VPN) stops interception on the network. **Key management** creates, stores, rotates and retires keys, which keeps control over who can decrypt data.

Meeting data sovereignty requirements

(1) Identify which laws apply to each type of data, such as customer data, payment data and logs, because different data may have different location rules. (2) Choose cloud regions inside the permitted country or region. (3) Restrict replication and backups to approved regions so that copies do not leave the country by accident. (4) Use policies that block creation of resources in other regions, which stops mistakes by developers or administrators. (5) Keep encryption keys in the same jurisdiction and under the institution's control to limit foreign access to the data. (6) Check support access, analytics tools and third-party services, because data can be exposed through tools that process it elsewhere. (7) Audit and report location settings regularly to provide proof to regulators.

Example: protecting one online fund transfer

At **login**, a password plus a one-time code (MFA) prevents account takeover with stolen passwords. When the request is **sent**, a TLS encrypted connection prevents interception of transaction details. During **authorization**, role and limit checks apply, and unusual amounts need extra approval, which reduces unauthorised or fraudulent transfers. The record is **stored** in an encrypted database in an approved region, which protects against data theft and legal non-compliance. Immutable **logs** of login, approval and transfer support audits and prevent denial of actions. Finally, **monitoring rules** and alerts watch for strange patterns, so fraud is discovered early.

Conclusion: Compliance controls, auditing, logging, monitoring, encryption, and appropriate data-location policies help financial institutions protect sensitive data and meet regulatory obligations.

5. Containerization, Virtual Machines, and Kubernetes

Containerization packages an application and its required libraries and dependencies into a lightweight, portable unit called a container. Containers help developers deploy applications consistently across different environments.

1. Containers vs. virtual machines:

Feature	Containers	Virtual Machines (VMs)
Architecture	Share the host operating system kernel	Run a guest operating system on virtualized hardware
Resource usage	Usually lower	Usually higher
Startup time	Generally faster	Generally slower
Isolation	Process-level isolation	Separate guest OS provides a stronger isolation boundary
Portability	Portable across compatible container environments	Portable across compatible virtualization platforms

2. **Performance:** Containers generally have less overhead because they share the host OS kernel. VMs require additional resources for their guest operating systems.

3. **Orchestration using Kubernetes:** Kubernetes automates container deployment, scaling, service discovery, and recovery. It can restart failed containers, distribute workloads, and increase or decrease application replicas according to demand.

4. **Portability:** Containers package application dependencies consistently, allowing applications to run across compatible development systems, on-premises servers, and different cloud providers with fewer environment-related changes.

5. **Microservices example:** Consider an online shopping application containing separate services for user login, product catalog, payments, and order management. Each service can run in its own container. Kubernetes manages these containers and scales the product catalog service independently when customer traffic increases.

Architecture of virtual machines and containers (diagram)

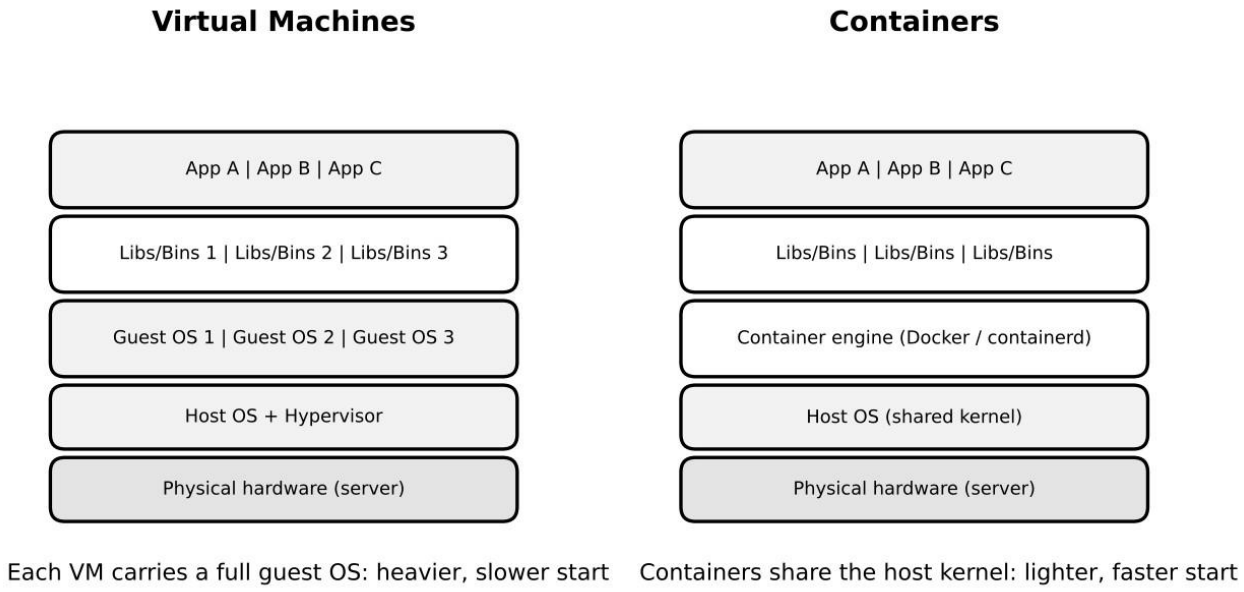


Figure 5.1: Layers of a virtual-machine server compared with a container server

Each virtual machine contains a full guest operating system on top of a hypervisor, so a server can run fewer VMs and each one takes longer to start. Containers share the host kernel and include only the application and its libraries, so many more containers fit on the same server and they start in seconds.

Choosing between virtual machines and containers

Containers are the better choice for running many small, scalable services and for continuous integration with frequent releases (DevOps), because they are light, quick to start and the same image moves from testing to production. Virtual machines are better when different operating systems (for example Linux and Windows) must run on one server, when untrusted or highly sensitive workloads need a stronger isolation boundary, or when an old application must be migrated with no changes. In practice many organisations combine both: containers run inside virtual machines provided by the cloud, which gives fast deployment and strong separation between customers.

From code to running application (flow chart)

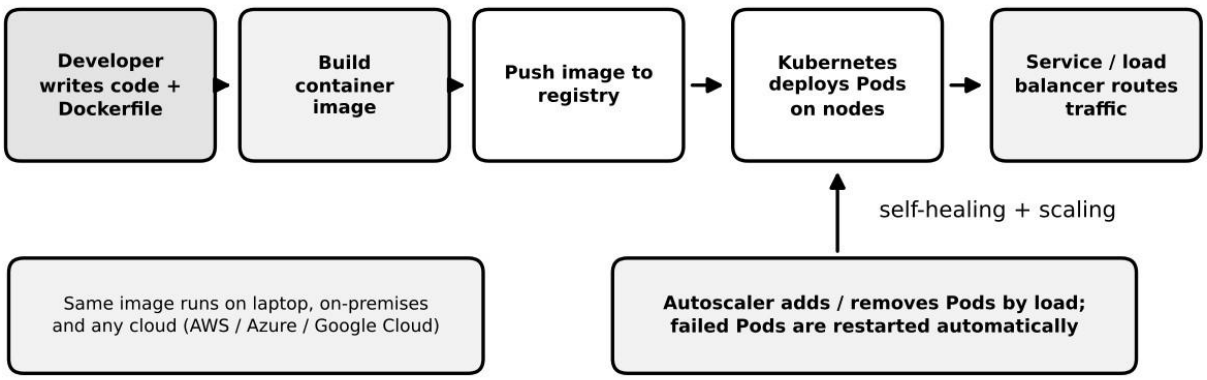


Figure 5.2: Build, ship and run workflow using containers and Kubernetes

Flow explained: (1) The developer writes the code and a Dockerfile that lists the base image and dependencies. (2) The image is built. (3) The image is pushed to a registry. (4) Kubernetes pulls the image and starts Pods (groups of one or more containers) on worker nodes. (5) A Service and load balancer send user traffic to the healthy Pods. (6) The autoscaler changes the number of Pods according to load, and failed Pods are restarted automatically.

Important Kubernetes terms

A **Pod** is the smallest deployable unit and holds one or more containers that share network and storage. A **Node** is a worker machine (physical or virtual) on which Pods run, and a **Cluster** is a group of nodes managed together by the Kubernetes control plane. A **Deployment** describes the desired number of Pod copies (replicas) and handles rolling updates and rollbacks. A **Service** gives a stable address to a set of Pods and balances traffic between them, while an **Ingress** routes external web traffic to the correct Service. The **Horizontal Pod Autoscaler** raises or lowers the number of Pods based on metrics such as CPU use, and **ConfigMaps and Secrets** store settings and sensitive values outside the container image.

Simple examples of a Dockerfile and a Kubernetes Deployment

Dockerfile for a small web service:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./ RUN
npm install --production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Kubernetes Deployment that keeps 3 copies of the product catalog service running:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-catalog
spec:
  replicas: 3
selector:
matchLabels:
      app: product-catalog
template:     metadata:
labels:
        app: product-catalog     spec:
containers:       - name: catalog
image: myregistry/product-catalog:1.0
ports:         - containerPort: 3000
```

Portability across different environments

The same container image can run on a developer laptop (Docker Desktop or a local Kubernetes tool such as minikube), on an on-premises Kubernetes cluster, and on managed services such as Amazon EKS, Azure Kubernetes Service (AKS) and Google Kubernetes Engine (GKE). Only environment settings such as network, load balancer and storage classes change; the application itself does not. Because the image carries its own libraries and settings, this reduces the “it works on my machine” problem and lowers the risk of being locked into a single cloud provider.

Microservices-based online shopping application

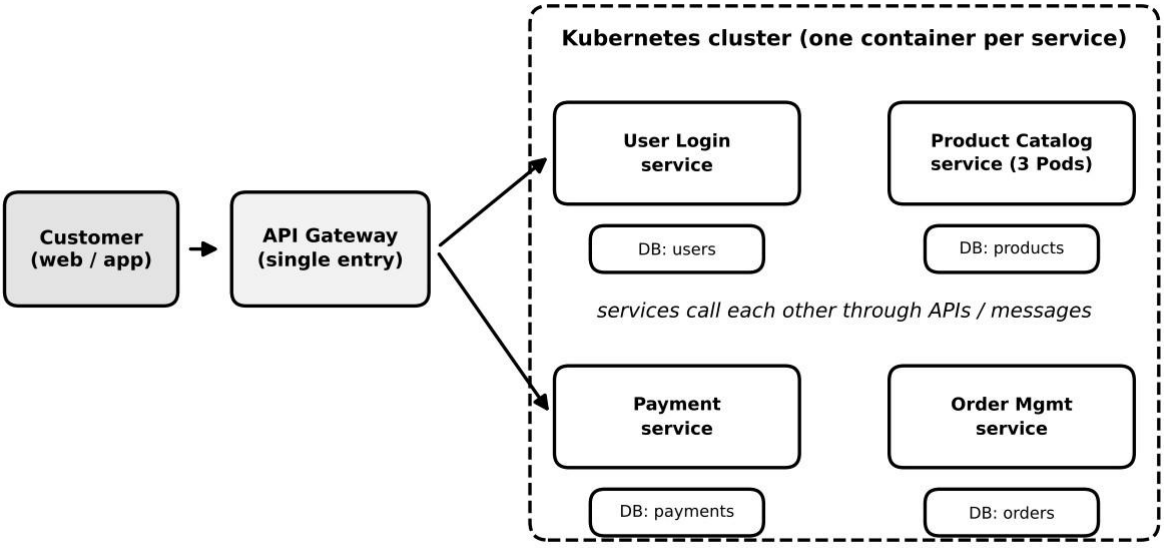


Figure 5.3: Online shopping application split into microservices running as containers in a Kubernetes cluster

In this example, **User Login** handles registration, sign-in and profiles using its own users database. **Product Catalog** handles the product list, search, prices and images, and is scaled the most during sales because browsing is the heaviest load. **Payment** handles payment requests and confirmation, and is scaled carefully and kept highly secure. **Order Management** handles cart, orders and delivery status, and scales with the number of purchases. Each service has its own data store.

Benefits: each service can be updated, scaled or fixed without stopping the others; a failure in one service does not stop the whole application; and different teams can work on different services in parallel. **Challenges:** more services mean more network communication and more to monitor; data must be kept consistent between separate databases; and good tools for logging, tracing and orchestration are needed, which Kubernetes helps to provide.

Conclusion: Containers offer efficient resource usage and portability, while Kubernetes simplifies deployment, scaling, and management of containerized applications across cloud environments.